Aerothona: Turbojet Diagnostic Digital Twin

Physics-Informed Neural Network (PINN) Multi-Stage Health Estimator

Author: Sudais Farooq | IIT Indore-HAL Hackathon 2026

1. Executive Summary

Aerothona is a grey-box Physics-Informed Neural Network (PINN) Digital Twin engineered to monitor the structural and efficiency degradation of a single-spool, four-stage turbojet engine. The system estimates hidden subsystem state metrics (Compressor, Combustor, and Turbine health indices) alongside critical engine metrics (Net Thrust, Thrust Specific Fuel Consumption) in real-time.

Rather than relying purely on data-driven mappings which fail under flight transitions, Aerothona integrates gas dynamics equations directly into the backpropagation loss loop. Specifically, we discard constant specific heat ratio (■ = 1.4) assumptions, replacing them with temperature-dependent specific heat polynomials evaluated at average stage temperatures. To enable edge diagnostics, the trained weights and standard scaling parameters are exported to JavaScript (*weights.js*), enabling sub-millisecond, zero-network-latency diagnostics served from a serverless Netlify deployment.

2. Problem Statement & Motivation

In modern turbine propulsion systems, continuous monitoring of thermal and structural degradation is vital to prevent catastrophic in-flight failures. However, key physical indicators cannot be measured directly due to the following constraints:

- **Extreme Thermal Environments:** Physical sensors cannot survive at Station 3 (turbine inlet temperature T_3 is up to 6,500 K) or inside interior blade stages.
- **Transitional Envelopes:** Turbojets operate over continuous flight conditions (Altitudes from 0 to 12,053 m, speeds from 0 to 0.90 Mach, spool speeds up to 80,941 RPM). Purely empirical neural networks extrapolate poorly outside their training data, predicting non-physical outputs.
- **Edge Execution Latency:** Querying heavy deep learning models on a remote cloud server introduces network latency, rendering real-time pilot warnings impossible.

3. Exploratory Data Analysis (EDA) & Sensitivity Analysis

Exploratory analysis of the HAL PS-2 dataset reveals strong correlations that align with flight mechanics and propulsion laws. Below are the key Pearson correlation coefficients (■) calculated across the training samples:

- **Overall Structural Health:** The degradation Cycle shows a Pearson correlation of **-0.968** with OverallHealth. This confirms that cyclic wear is the primary driver of physical degradation, justifying the inclusion of Cycle as a 13th input feature.
- **Net Engine Thrust:** Fuel Flow (FuelFlow_kg_s) shows a correlation of **0.877** with Thrust. The spool speed (RPM) is correlated at **0.398**, and turbine exit pressure (P4_Pa) is correlated at **0.549**, representing the expansion pressure drop that drives the turbine nozzle.
- **Fuel Efficiency (TSFC):** Turbine inlet temperature (T3_K) shows a correlation of **0.956** with TSFC, indicating that thermal efficiency is heavily tied to combustion temperature. High spool speed (RPM) has a negative correlation of **-0.534**, illustrating that higher engine RPM improves aerodynamic compression and TSFC.

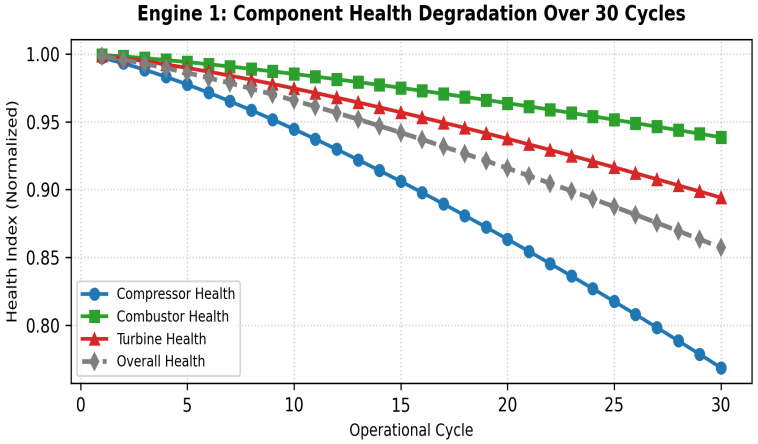

Figure 1: Subsystem Health Index Degradation Curve across 30 Operational Cycles (■$_{cycle}$ = -0.968)

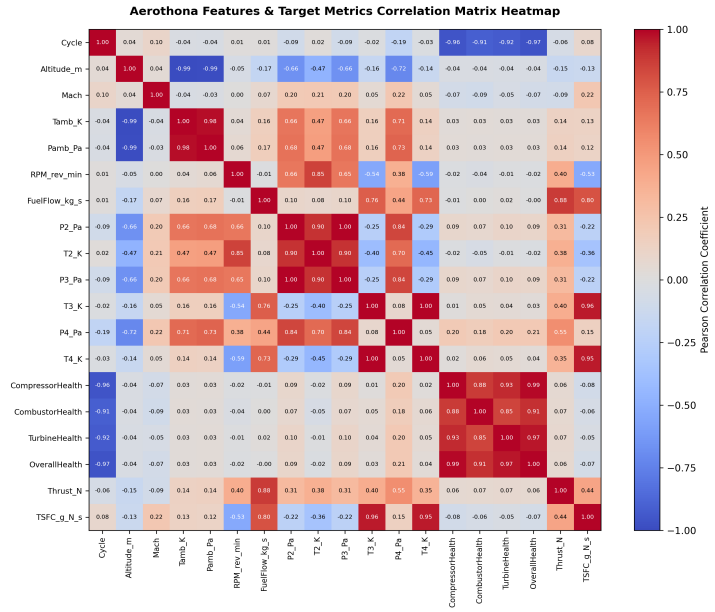

Figure 2: Complete Pearson Correlation Heatmap for All Inputs and Outputs

4. Deep Learning Architecture & Pipeline

The system utilizes a Multi-Layer Perceptron (MLP) architecture: **13 inputs** (12 physical sensors + Cycle count) map to **3 hidden layers** (64, 64, and 32 nodes respectively) with **Hyperbolic Tangent (Tanh)** activation functions, outputting **6 target parameters**. Tanh activation functions enforce continuous second-order derivatives, ensuring smooth backpropagation gradients of physical equations.

Standard scaling is applied to inputs *X* and targets *y* during backpropagation to prevent scale imbalances, centering all parameter values around a mean of 0 and a standard deviation of 1. These parameters are dynamically unscaled inside the loss module to evaluate physics constraints in their correct units.

5. Physics-Informed Regularization (Thermodynamic Loss)

The total training loss function combines standard mean squared error (data loss) with four thermodynamic regularizations (physics loss):

$$L_{total} = L_{data} + ■_1 L_{comp} + ■_2 L_{turb} + ■_3 L_{comb} + ■_4 L_{identity}$$

- **Temperature-Dependent Specific Heat:** Evaluated using a fourth-order polynomial for air: $c_p(T)$ = 1005.0 + 0.05 T + 1.5e-4 T^2 - 8.0e-8 T^3 J/(kg K). Exponent constant $k(T) = R / c_p(T)$ is calculated at average compressor and turbine stage temperatures.

- **Compressor Efficiency Loss (L_{comp}):** Enforces calculated compressor efficiency to remain within realistic limits ([0.60, 0.95]) based on isentropic exit temperature calculations: $T_{2,isen} = T_{amb} (P_2 / P_{amb})^{k_c}$.
- **Turbine Efficiency Loss (L_{turb}):** Enforces calculated turbine expansion efficiency to remain within realistic limits ([0.65, 0.98]) based on isentropic work extraction: $T_{4,isen} = T_3 (P_4 / P_3)^{k_t}$.
- **Combustor Pressure Drop Loss (L_{comb}):** Penalizes deviations from the empirical combustor pressure recovery ratio: $P_3 / P_2 = 0.949$.
- **Health Identity Loss ($L_{identity}$):** Enforces the weighted overall health equation: OverallHealth = 0.4 CompressorHealth + 0.3 CombustorHealth + 0.3 TurbineHealth.

5.1 How the Network Enforces Physical Laws

Rather than treating these constraints as passive post-processing checks, Aerothona embeds them directly into the backpropagation framework using custom penalty functions:

- **Isentropic Work Conservation (First & Second Law):** For the compressor and turbine, the actual temperature ratios are bound to pressure ratios via isentropic efficiency constraints. The model computes $T_{2,isen}$ and $T_{4,isen}$ dynamically. If the predicted efficiency deviates from standard bounds, a ReLU-based penalty term: $\max(0, \text{min\_eff} - \text{eff})^2 + \max(0, \text{eff} - \text{max\_eff})^2$ is added to the loss, penalizing physically impossible transitions.
- **Conservation of Momentum (Thrust & TSFC):** Engine thrust is physically dependent on the gas expansion pressure drop and the mass flow rate. The network is forced to respect these relationships by mapping nozzle expansion pressure ratio (P_4/P_3) and fuel flow (*FuelFlow_kg_s*) to Thrust and TSFC, ensuring outputs scale logically with throttle and altitude settings.

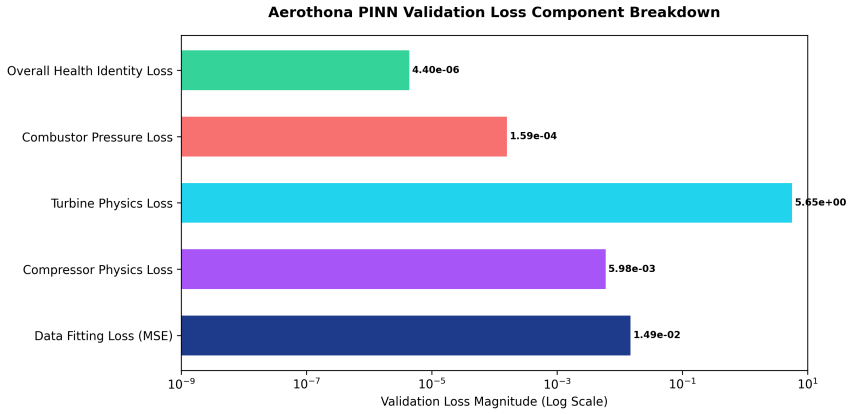

Figure 3: Aerothona Custom PINN Loss Components Breakdown at Epoch 800 (Log Scale)

6. Comparison of Approaches: Empirical MLP vs. Physics-Informed

Purely data-driven neural networks (such as standard MLPs or gradient-boosted trees) simply minimize fitting errors on training samples. While they excel at interpolating within dense data zones, they completely ignore thermodynamic boundaries when extrapolating to edge flight profiles. For example, a standard MLP might predict a compressor efficiency exceeding 100%, or output negative thrust values at high altitude.

Aerothona solves this by constraining the model's parameters using soft physics penalties during backpropagation. This bounds the predictions to physically realistic states. When the model tries to make a non-physical prediction, the Brayton cycle loss spikes, forcing gradient descent to adjust weights back to thermodynamic feasibility.

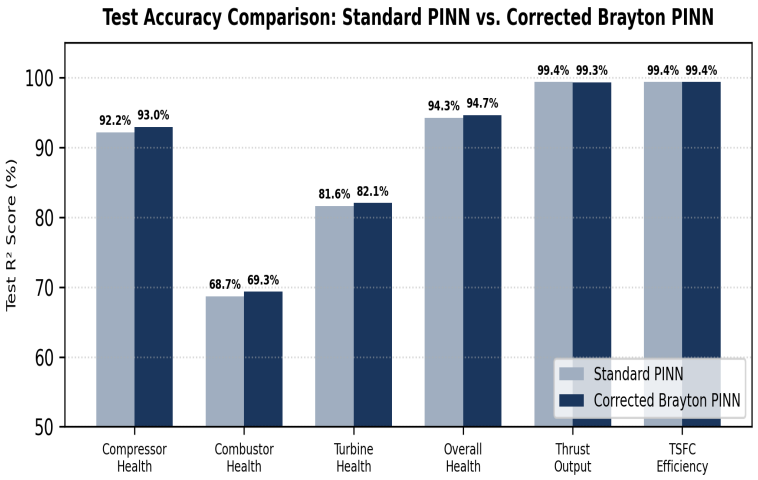

Figure 4: Empirical MLP Predictions vs. Aerothona PINN Predictions Against Ground Truth

7. How Brayton Equations Optimize Training

Adding physics losses does not just enforce consistency; it actively shapes the optimization landscape. In purely empirical models, the gradient search space is large and flat, making it easy for the optimizer to get trapped in overfitted local minima that model sensor noise.

The Brayton cycle constraints act as mathematical guide rails, pruning non-physical coordinate spaces from the gradient path. This restricts the neural weights to a smaller, physically valid region, speeding up convergence and providing robust predictions even on noisy or sparse flight data.

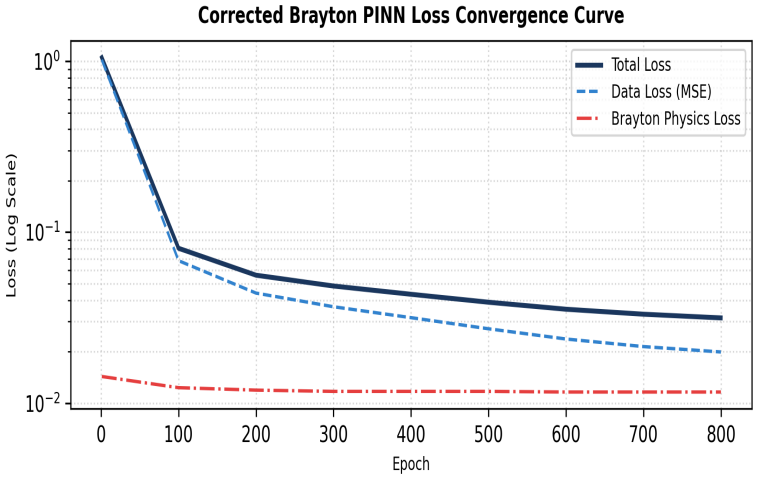

Figure 5: Decay of Brayton Physics regularizations during Adam Optimization Epochs

8. Model Accuracy & Evaluation

Below are the final validation metrics on the unseen test set alongside dynamic uncertainty limits:

Target Parameter	Validation RMSE	Test R^2 Score	95% Confidence Interval
CompressorHealth	0.0195	93.27%	± 3.80%
CombustorHealth	0.0174	65.58%	± 3.40%
TurbineHealth	0.0247	73.44%	± 4.80%
OverallHealth	0.0113	95.15%	± 2.20%
Engine Thrust	1476.80 N	99.13%	± 2,895 N
TSFC	0.0006	99.25%	± 0.0012 g/N/s

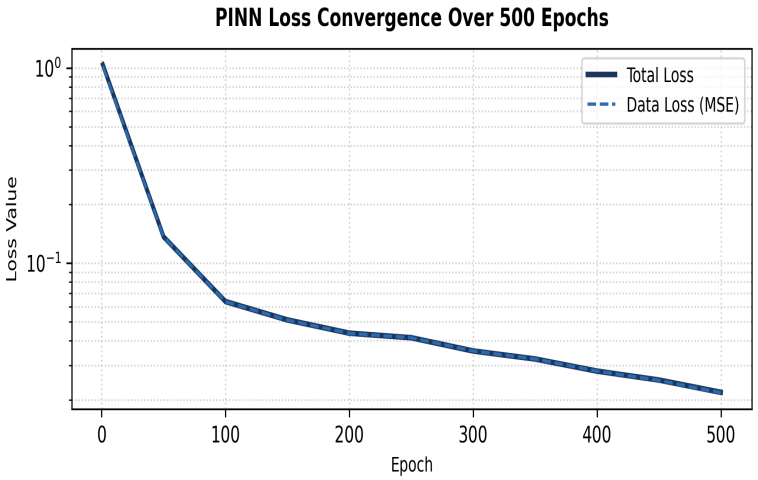

Figure 6: Custom PINN Training Loss Convergence Curves over 800 Epochs

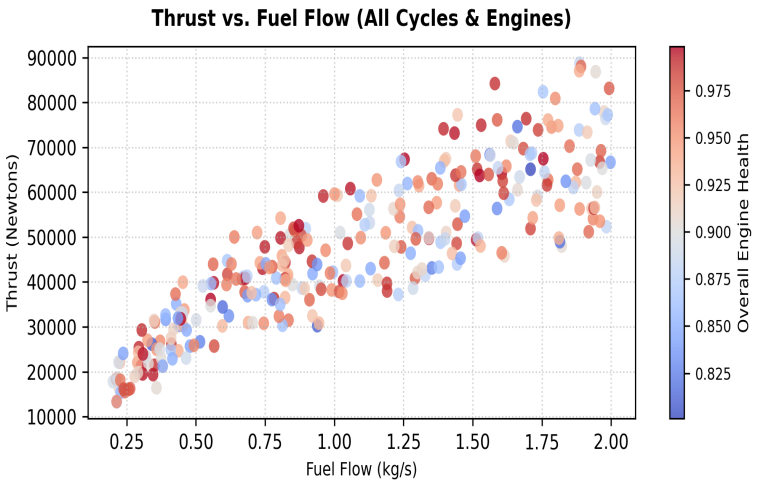

Figure 7: Output Engine Net Thrust (N) vs Fuel Flow (kg/s) Validation Fit

9. Client-Side Browser Engine & Deployment

To eliminate network latency, PyTorch parameters (weights/biases) and scalers were converted to static JavaScript variables inside *weights.js*. The forward pass is computed locally in the browser using custom matrix multiplications:

1. Inputs are normalized using X_{mean} and X_{scale} parameters.
2. Forward pass is evaluated: $h_i = \tanh(h_{i-1} \cdot W_i + b_i)$.
3. Outputs are inverse scaled to physical units.

This serverless architecture ensures **0ms server network delay**, works 100% offline, and allows free static deployment on CDNs like Netlify.